CMR UNIVERSITY
Private University Established in Karnataka State by Act No. 45 of 2013

CAPSTONE PROJECT • SECOND REVIEW (REVIEW-2) • AI & ML

CodeArchaeologist

An AST-Guided, LLM-Powered System for Legacy Code Comprehension & Automated Documentation

TEAM MEMBERS

VISHAL KUMAR JANGIR : 23BBTCS280
SYED MUDASSIR : 23BBTCS258
JUNAID PASHA : 23BBTCS257
GAURAV SHARMA : 23BBTCS306

GUIDE

Mr. Sheik Jamil Ahmed(CSE)

Department of CSE, School of Engineering & Technology

Presentation Outline

CMR UNIVERSITY

1 Problem Statement

2 Final Objectives

3 Literature Survey

4 Identified Gaps & Challenges

5 System Architecture

6 Detailed Design

7 Workflow Overview

8 Technology Stack

9 Implementation Plan

10 Expected Outcomes

11 Conclusion & Future Scope

12 References

Problem Statement

The core problem

Understanding existing code — not writing it — is where developers lose the most time.

Developers spend the majority of their time reading and understanding existing code rather than writing new code. In large or ageing systems this code is frequently undocumented, and the engineers who originally wrote it may no longer be available. As a result, developers must manually trace logic across many files to answer even simple questions — such as what a module does or how components depend on one another. This is a well-known bottleneck that slows onboarding, delays development, and increases the likelihood of defects.

Traditional static-analysis tools extract accurate structure — abstract syntax trees, call graphs, and complexity metrics — but cannot explain code in natural language. Large Language Models explain code fluently, yet used alone they may hallucinate and struggle with large repositories. There is a clear need for a system that combines the precision of static analysis with the expressiveness of LLMs to make undocumented codebases understandable, documented, and searchable.

Final Objectives

Automated Code Comprehension

Generate clear, plain-English explanations of files, classes, and functions.

Architecture Visualization

Build call graphs and dependency diagrams that reveal system structure.

Documentation & Quality Analysis

Produce module-level docs, complexity metrics, and dead-code reports.

Natural-Language Code Q&A

Answer questions about the codebase using semantic search over the code.

Grounded, Trustworthy AI

Feed the LLM structured static-analysis output to reduce hallucination.

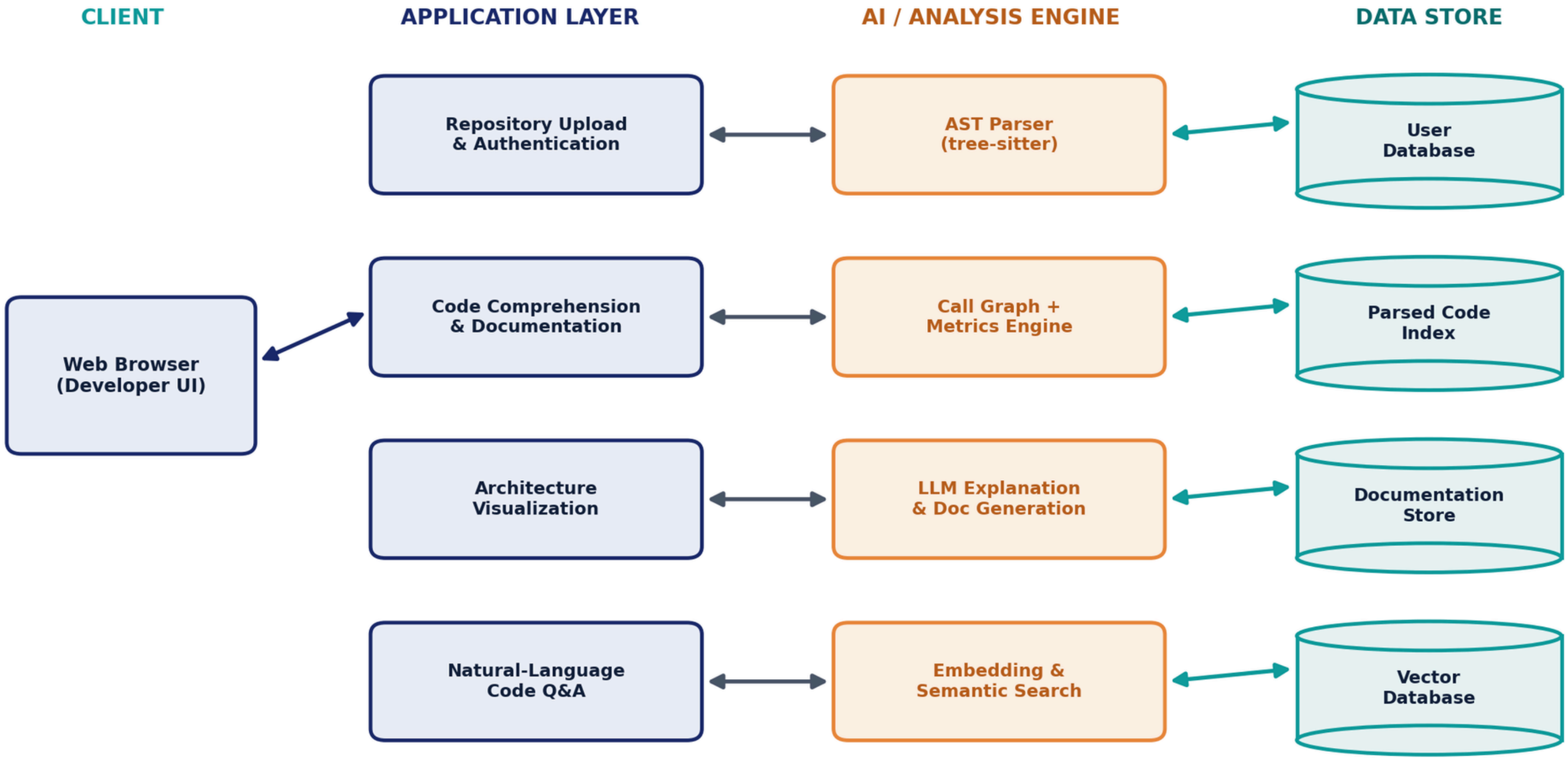
Literature Survey

Ref / Year	Work	Key Contribution
Feng et al., 2020	**CodeBERT**	Pre-trained model bridging code and natural language.
Chen et al., 2021	**Codex**	LLMs trained on code for generation and comprehension.
Guo et al., 2021	**GraphCodeBERT**	Adds data-flow structure to code representations.
Hu et al., 2018	**Deep Code Comment Gen.**	Automatic comment / documentation generation.
Ahmad et al., 2020	**Code Summarization**	Transformer-based source-code summarization.
Allamanis et al., 2018	**ML for Big Code (Survey)**	Foundations of machine learning over source code.

Identified Gaps & Challenges

Gap / Challenge in Existing Work	How CodeArchaeologist Addresses It
Static-analysis tools give structure & metrics but no human-readable explanation.	**Uses an LLM layer to turn structure into plain-English explanations.**
LLM-only tools hallucinate and hit context limits on large repositories.	**Grounds the LLM in verified AST/call-graph context for reliable output.**
Documentation is manual and quickly goes stale as code changes.	**Regenerates docs directly from source — change-aware, never outdated.**
No easy way to search a codebase by meaning, only by keywords.	**Adds embedding-based semantic Q&A over the actual code.**
Existing solutions rarely combine both structural and semantic understanding.	**Unifies classical program analysis with modern generative AI.**

System Architecture



System Modules

Module 1

Code Ingestion

Accepts a repo/folder and prepares source files for analysis.

Module 2

Static Analysis

Parses code into ASTs and resolves cross-file dependencies.

Module 3

Metrics & Quality

Call graph, cyclomatic complexity, and dead-code detection.

Module 4

LLM Explanation

Generates grounded explanations and documentation.

Module 5

Visualization

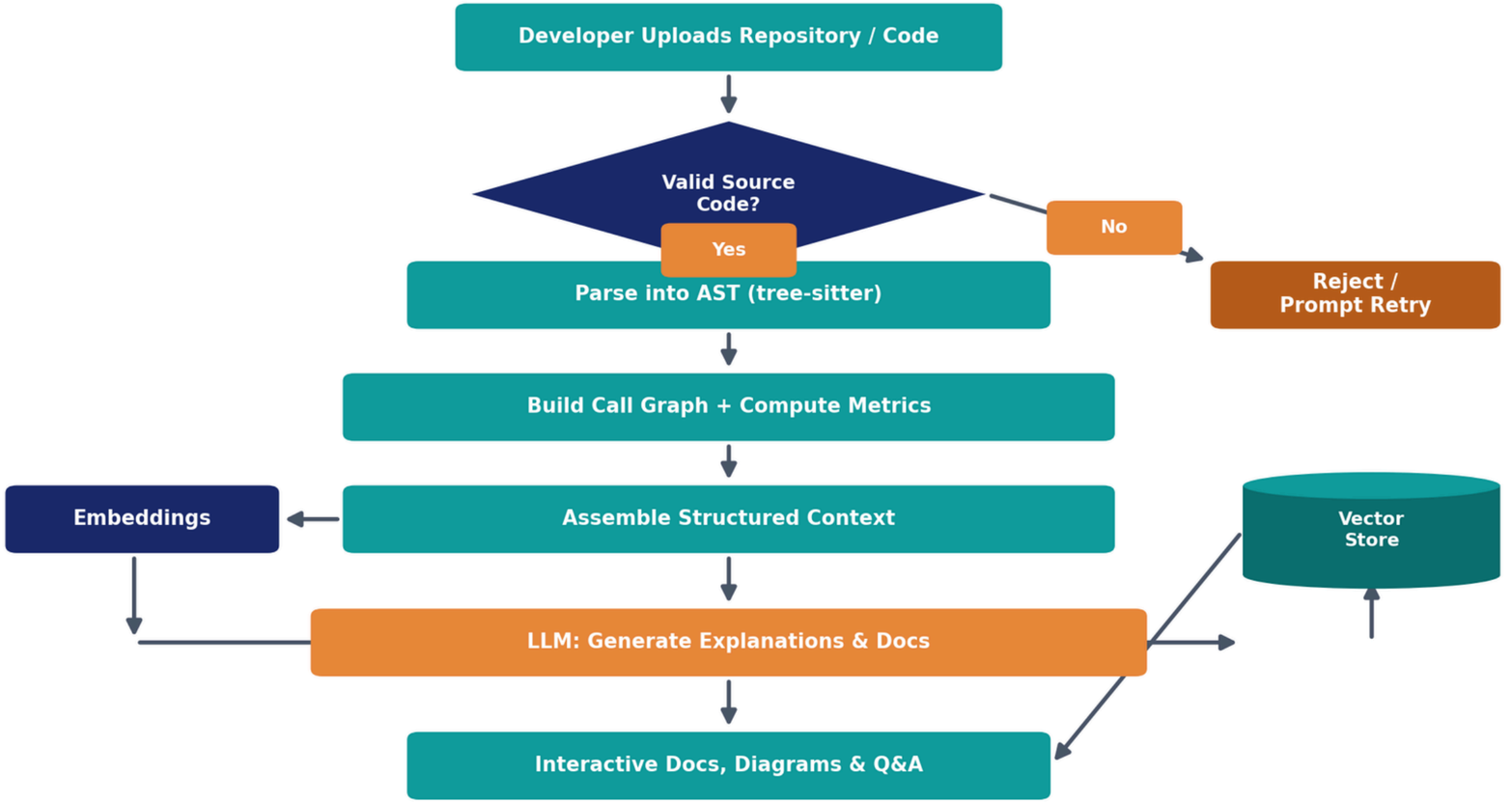
Renders interactive architecture & dependency diagrams.

Module 6

Semantic Q&A

Embedding-based natural-language search over the code.

Data Flow Diagram



Data & Output Design

The analysis contract

- The static-analysis engine emits one structured JSON object per codebase.
- This schema is the single contract consumed by the LLM, visualization, and Q&A modules.
- Designing it early keeps the layers cleanly decoupled and testable.

analysis_output.json

```
{
  "repository": { "name", "files", "total_loc" },
  "modules": [
    { "path", "functions", "classes",
      "imports", "loc" } ],
  "functions": [
    { "name", "file", "line_range",
      "complexity", "calls" } ],
  "call_graph": { "nodes", "edges" },
  "metrics": { "avg_complexity",
    "dead_code", "loc_summary" }
}
```

Workflow Overview

CMR UNIVERSITY

1

Ingest

Developer uploads a repository or source folder.

2

Parse (AST)

Extract functions, classes & dependencies.

3

Analyze

Build call graph & compute quality metrics.

4

Explain (LLM)

Generate grounded explanations & documentation.

5

Interact

Query the codebase in natural language.

Static analysis supplies clean, structured context to the LLM — the key to grounded, low-hallucination output.

Technology Stack

Python

Core language

tree-sitter

AST parsing

NetworkX

Call graph & metrics

LLM (Claude/GPT)

Explanation & docs

Sentence-Transformers

Embeddings

FAISS / ChromaDB

Vector search

FastAPI

Backend API

React + Mermaid.js

Frontend & diagrams

Implementation Plan

Phase 1 — Analysis Engine

1. Requirement study & literature survey
2. AST parsing (tree-sitter) for Python
3. Call graph, complexity & dead-code metrics
4. CLI prototype + structured JSON output

Phase 2 — AI & Interface

1. LLM explanation & documentation layer
2. Embeddings + semantic code Q&A
3. Web UI with architecture diagrams
4. Integration, evaluation & deployment

Current status: Phase 1 in progress — static-analysis engine under active development.

Expected Outcomes

Deliverables

- Plain-language documentation for any supported codebase.
- Interactive architecture and call-graph diagrams.
- Complexity and dead-code reports for maintainability.
- Natural-language Q&A over the codebase.
- Reduced onboarding time for new developers.

Grounded

AI output verified against real code structure, lowering hallucination.

Faster

Cuts the time spent understanding unfamiliar code.

Always current

Docs regenerate as the code changes — never stale.

Conclusion & Future Scope

Conclusion

CodeArchaeologist unites classical program analysis with modern LLMs to turn opaque, undocumented codebases into clear explanations, living documentation, and searchable knowledge. By grounding AI in verifiable structure, it delivers reliable insight — reducing comprehension effort and improving maintainability across real-world software.

Future Scope

- Support for more languages (Java, C++, JS).
- IDE plugins for in-editor explanations.
- Automated hallucination & faithfulness evaluation.
- Team knowledge base with change-aware docs.

References

[1] Z. Feng et al., “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” Findings of EMNLP, pp. 1536–1547, 2020.

[2] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.

[3] D. Guo et al., “GraphCodeBERT: Pre-training Code Representations with Data Flow,” ICLR, 2021.

[4] X. Hu, G. Li, X. Xia, D. Lo, Z. Jin, “Deep Code Comment Generation,” IEEE/ACM ICPC, pp. 200–210, 2018.

[5] W. U. Ahmad, S. Chakraborty, B. Ray, K. Chang, “A Transformer-based Approach for Source Code Summarization,” ACL, pp. 4998–5007, 2020.

[6] M. Allamanis, E. T. Barr, P. Devanbu, C. Sutton, “A Survey of Machine Learning for Big Code and Naturalness,” ACM Computing Surveys, vol. 51, no. 4, pp. 1–37, 2018.

Presentation













Thank You

Questions & Discussion

Project: CodeArchaeologist — Second Review
Guide: **Mr. Sheik. J**amil Ahmed(CSE)